

Escenarios de Calidad (ASRs) — Solventa

e Historias de Usuario

Versión 1.1 · Semana 3 · 20 de agosto de 2026

Integrante	Rol
Jazmin Natalia Córdoba	Coordinadora — Usabilidad
Juan Esteban Mejía	Integración y API
Miguel Alejandro Gómez	Rendimiento y Seguridad
Angie Natalia Arandio	Unitarias y Módulo

Sección 1: Escenarios de Atributos de Calidad (ASR)

Los siguientes 9 escenarios de calidad fueron definidos usando el formato SEI/Bass (6 campos: fuente del estímulo, estímulo, ambiente, artefacto, respuesta y medida de respuesta). Cada ASR incluye además la frase narrativa 'Como <fuente>, cuando <estímulo>, dado que el ambiente es <ambiente>, quiero que <respuesta>, esto debe suceder <medida de respuesta>', está vinculado a una historia de usuario de arquitectura en Jira, y tiene sus dependencias explícitas con otras historias del backlog.

ASR	Atributo de calidad	Historia Jira	Medida clave
ASR-1.1	Latencia	SOL-28	p95 < 200ms con 500 req/min
ASR-1.2	Latencia	SOL-37	p95 < 200ms desde caché, 0% pérdida

ASR	Atributo de calidad	Historia Jira	Medida clave
ASR-2.1	Modificabilidad	SOL-20	< 4 horas-hombre, 0 cambios en core
ASR-2.2	Modificabilidad	SOL-21	≤ 2 semanas, 0 cambios en motor rating
ASR-3.1	Disponibilidad	SOL-29	Recuperación < 30s, ≥ 99.9% disp.
ASR-3.2	Disponibilidad	SOL-43	RTO ≤ 10 min, RPO ≤ 30s
ASR-3.3	Disponibilidad	SOL-44	100% consultas offline, sync ≤ 10s
ASR-4.1	Seguridad	SOL-23	100% TLS 1.3, 0 PII en texto plano
ASR-4.2	Seguridad	SOL-45	Alerta modificación no autorizada < 1s

ASR-1.1 — Latencia — Scoring en carga normal

Atributo: Rendimiento / Latencia

Campo SEI/Bass	Valor
Fuente del estímulo	Socio de distribución (banco originador de crédito hipotecario)
Estímulo	Solicitud de cotización de seguro de vida hipotecario con perfilamiento Open Finance
Ambiente	Sistema operando en condiciones normales (500 req/min)
Artefacto	Servicio de Cotización & Rating + Servicio de Perfilamiento
Respuesta	El sistema calcula el perfil de riesgo combinando señales Open Finance y Open Data y devuelve la prima estimada
Medida de respuesta	95% de solicitudes respondidas en < 200ms; p99 < 400ms

Escenario narrativo:

Como socio de distribución (banco originador de crédito hipotecario), cuando se solicita una cotización de seguro de vida hipotecario con perfilamiento Open Finance, dado que el ambiente es un sistema operando en condiciones normales (500 solicitudes por minuto), quiero que el sistema calcule el perfil de riesgo combinando señales Open Finance y Open Data y devuelva la prima estimada, esto debe suceder en menos de 200ms para el 95% de las solicitudes, con p99 menor a 400ms.

Historia de usuario en Jira:

HU-ARQ-07 — Optimización de Latencia Motor de Scoring —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-28>

Dependencias:

- HU-ARQ-05 (SOL-26) — Persistencia PostgreSQL + Redis: el caché Redis es necesario para alcanzar p95 < 200ms
- HU-ARQ-08 (SOL-37) — Event Bus Kafka: la cotización publica eventos de dominio de forma asíncrona
- FE-01 (SOL-3) — Cotización web: es el journey de usuario que activa este ASR

ASR-1.2 — Latencia — Scoring con proveedor Open Finance degradado

Atributo: Rendimiento / Latencia + Disponibilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Cliente asegurado durante campaña masiva de socio (aerolínea)
Estímulo	5.000 solicitudes de scoring concurrentes con proveedor Open Finance respondiendo lento (>700ms)
Ambiente	Sistema bajo carga pico (10× tráfico normal), circuit breaker activo
Artefacto	Servicio de Perfilamiento + Caché Redis + Circuit Breaker
Respuesta	El sistema usa perfil de riesgo cacheado cuando Open Finance supera el timeout, operando en modo degradado elegante
Medida de respuesta	95% de solicitudes respondidas en < 200ms usando caché; 0% de solicitudes perdidas por timeout externo

Escenario narrativo:

Como cliente asegurado durante una campaña masiva de un socio (por ejemplo, una aerolínea), cuando se generan 5.000 solicitudes de scoring concurrentes con el proveedor Open Finance respondiendo lento (más de 700ms), dado que el ambiente es un sistema bajo carga pico (10 veces el tráfico normal) con el circuit breaker activo, quiero que el sistema use el perfil de riesgo cacheado cuando Open Finance supere el timeout, operando en modo degradado elegante, esto debe suceder respondiendo el 95% de las solicitudes en menos de 200ms usando caché, con 0% de solicitudes perdidas por timeout externo.

Historia de usuario en Jira:

HU-ARQ-08 — Event Bus con Kafka y Degradación Elegante —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-37>

Dependencias:

- ASR-1.1 (SOL-28) — la latencia base debe estar resuelta antes que la degradación elegante
- HU-ARQ-05 (SOL-26) — Redis caché: fuente de datos durante la degradación
- HU-ARQ-02 (SOL-20) — Abstracciones pasarelas: el adaptador Open Finance sigue el mismo patrón
- FE-01 (SOL-3) — Cotización web: journey afectado por la degradación

ASR-2.1 — Modificabilidad — Nueva pasarela de pagos regional

Atributo: Modificabilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Equipo de desarrollo (desarrollador backend)
Estímulo	Integración de nueva pasarela de pagos para el mercado mexicano
Ambiente	Sistema en producción, otros servicios operando normalmente
Artefacto	Adaptador de Pagos (capa de adaptadores externos)
Respuesta	El desarrollador implementa el nuevo adaptador sin modificar el servicio de Pagos & Recaudo ni ningún otro servicio del núcleo
Medida de respuesta	Modificación + pruebas + despliegue completados en < 4 horas-hombre; cero cambios en servicios del núcleo de negocio

Escenario narrativo:

Como equipo de desarrollo (desarrollador backend), cuando se requiere integrar una nueva pasarela de pagos para el mercado mexicano, dado que el ambiente es un sistema en producción con otros servicios operando normalmente, quiero que el desarrollador implemente el nuevo adaptador sin modificar el servicio de Pagos & Recaudo ni ningún otro servicio del núcleo, esto debe suceder en menos de 4 horas-hombre de modificación, pruebas y despliegue, con cero cambios en los servicios del núcleo de negocio.

Historia de usuario en Jira:

HU-ARQ-02 — Abstracciones de Pasarelas de Pago (Adapter Pattern) —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-20>

Dependencias:

- FE-02 (SOL-4) — Suscripción y emisión: journey que usa el módulo de pagos
- HU-ARQ-04 (SOL-22) — Gestión de consentimientos: el pago requiere consentimiento activo del usuario
- HU-ARQ-03 (SOL-23) — Tokenización PII: las credenciales de pago deben tokenizarse

ASR-2.2 — Modificabilidad — Nuevo ramo de seguro

Atributo: Modificabilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Área de producto / negocio
Estímulo	Solicitud de lanzamiento de nuevo ramo (seguro de mascota) no contemplado en el diseño inicial

Ambiente	Sistema en producción con 5 ramos activos
Artefacto	Servicio de Cotización & Rating + módulo de configuración de ramos
Respuesta	El equipo configura el nuevo ramo mediante parámetros sin modificar la lógica central del motor de rating
Medida de respuesta	Nuevo ramo disponible en producción en ≤ 2 semanas-equipo; modificaciones confinadas a configuración y adaptadores

Escenario narrativo:

Como área de producto/negocio, cuando se solicita el lanzamiento de un nuevo ramo de seguro (por ejemplo, seguro de mascota) no contemplado en el diseño inicial, dado que el ambiente es un sistema en producción con 5 ramos activos, quiero que el equipo configure el nuevo ramo mediante parámetros sin modificar la lógica central del motor de rating, esto debe suceder en un plazo máximo de 2 semanas-equipo, con las modificaciones confinadas a configuración y adaptadores.

Historia de usuario en Jira:

HU-ARQ-01 — Parametrización Multitenant / Multirregión —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-21>

Dependencias:

- HU-ARQ-05 (SOL-26) — Persistencia PostgreSQL: el catálogo de ramos se almacena en BD
- FE-01 (SOL-3) — Cotización: el nuevo ramo debe aparecer en el flujo de cotización automáticamente
- FE-09 (SOL-31) — Portal de socios: los socios distribuidores ven el nuevo ramo vía API

ASR-3.1 — Disponibilidad — Caída del contenedor Redis

Atributo: Disponibilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Infraestructura interna (falla de contenedor)
Estímulo	Contenedor Redis se desconecta inesperadamente durante operación normal
Ambiente	Sistema en producción, usuarios activos realizando cotizaciones
Artefacto	BFF Web + Servicio de Cotización + Caché Redis
Respuesta	El sistema detecta la falla y redirige solicitudes al servicio de perfilamiento directo, manteniendo operación en modo degradado sin pérdida de transacciones activas
Medida de respuesta	Recuperación automática en < 30 segundos; disponibilidad mensual ≥ 99,9%; cero transacciones activas perdidas

Escenario narrativo:

Como infraestructura interna del sistema, cuando el contenedor de Redis se desconecta inesperadamente durante la operación normal, dado que el ambiente es un sistema en producción con usuarios activos realizando cotizaciones, quiero que el sistema detecte la falla y redirija las solicitudes al servicio de perfilamiento directo, manteniendo la operación en modo degradado sin pérdida de transacciones activas, esto debe suceder con recuperación automática en menos de 30 segundos, disponibilidad mensual mayor o igual a 99,9% y cero transacciones activas perdidas.

Historia de usuario en Jira:

HU-ARQ-06 — Tolerancia a Fallos y Failover (Circuit Breaker) —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-29>

Dependencias:

- HU-ARQ-05 (SOL-26) — Persistencia PostgreSQL + Redis: el circuit breaker hace fallback a PostgreSQL
- HU-ARQ-07 (SOL-28) — Latencia scoring: la latencia aumenta durante el failover pero debe permanecer < 500ms
- ASR-3.2 (SOL-43) — el mecanismo de failover de zona comparte lógica con el failover de Redis
- Relacionado pero DISTINTO de ASR-3.3 (SOL-44): ASR-3.3 cubre continuidad offline del cliente móvil, no la caída de Redis en backend

ASR-3.2 — Disponibilidad — Caída de zona de disponibilidad AWS

Atributo: Disponibilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Proveedor cloud (AWS)
Estímulo	Falla completa de la Zona de Disponibilidad A (us-east-1a) con instancias EKS activas
Ambiente	Sistema en producción, tráfico distribuido entre Zona A y Zona B en configuración activo-activo
Artefacto	EKS multi-zona + RDS PostgreSQL con réplica + Amazon MSK
Respuesta	El balanceador redirige todo el tráfico a Zona B; RDS promueve la réplica a primaria automáticamente; Kafka continúa operando con brokers en Zona B
Medida de respuesta	RTO ≤ 10 minutos; RPO ≤ 30 segundos; disponibilidad mensual ≥ 99,9%

Escenario narrativo:

Como plataforma Solventa expuesta a una falla del proveedor cloud (AWS), cuando ocurre una falla completa de la Zona de Disponibilidad A (us-east-1a) con instancias EKS activas, dado que el ambiente es un sistema en producción con tráfico distribuido entre la Zona A y la Zona B en configuración activo-activo, quiero que el balanceador redirija todo el tráfico a la Zona B, que RDS promueva automáticamente la réplica

a primaria y que Kafka continúe operando con brokers en la Zona B, esto debe suceder con un RTO menor o igual a 10 minutos, un RPO menor o igual a 30 segundos y disponibilidad mensual mayor o igual a 99,9%.

Historia de usuario en Jira:

HU-ARQ-09 — Autoescalado Horizontal EKS + Multi-AZ —
<https://proyectointegradorgrupo2.atlassian.net/browse/SOL-43>

Dependencias:

- HU-ARQ-06 (SOL-29) — Tolerancia a fallos: el failover de zona reutiliza el circuit breaker definido en este ASR
- HU-ARQ-05 (SOL-26) — RDS Multi-AZ: la BD debe estar configurada en multi-AZ para cumplir el RPO
- HU-ARQ-08 (SOL-37) — Kafka MSK: Amazon MSK también debe estar en multi-AZ para no perder eventos

ASR-3.3 — Disponibilidad — Continuidad offline del cliente móvil

Atributo: Disponibilidad

Campo SEI/Bass	Valor
Fuente del estímulo	Usuario móvil con pérdida de señal de datos/wifi
Estímulo	El dispositivo pierde conectividad de red mientras el usuario consulta sus pólizas activas en la billetera móvil
Ambiente	App móvil en uso activo, sesión ya autenticada, pólizas previamente sincronizadas
Artefacto	BFF Móvil + almacenamiento local cifrado (AsyncStorage) + módulo de sincronización
Respuesta	La app sirve las pólizas (número, vigencia, cobertura) desde el almacenamiento local cifrado sin requerir conexión, y sincroniza automáticamente los cambios pendientes al recuperar conectividad
Medida de respuesta	100% de las consultas de pólizas disponibles sin conexión; sincronización automática en ≤ 10 segundos al recuperar conectividad; datos offline con TTL de 24 horas (alerta de datos desactualizados al vencer)

Escenario narrativo:

Como usuario móvil, cuando el dispositivo pierde conectividad de red mientras consulta sus pólizas activas en la billetera móvil, dado que el ambiente es una app móvil en uso activo con sesión ya autenticada y pólizas previamente sincronizadas, quiero que la app sirva las pólizas (número, vigencia, cobertura) desde el almacenamiento local cifrado sin requerir conexión y sincronice automáticamente los cambios pendientes al recuperar conectividad, esto debe suceder garantizando el 100% de las consultas disponibles sin conexión, sincronización automática en menos de 10 segundos al recuperar conectividad, y datos offline con un TTL de 24 horas tras el cual se muestra una alerta de datos desactualizados.

Historia de usuario en Jira:

HU-ARQ-10 — Offline Sync BFF Móvil — <https://proyectointegradorgrupo2.atlassian.net/browse/SOL-44>

Dependencias:

- FE-06 (SOL-11) — Billetera de pólizas móvil: es la pantalla donde se activa este escenario
- Relacionado pero DISTINTO de ASR-3.1 (SOL-29): ambas son de disponibilidad/continuidad, pero cubren fallas diferentes (Redis en backend vs. pérdida de conectividad en el cliente móvil)

ASR-4.1 — Seguridad — Confidencialidad en transmisión de datos financieros Open Finance

Atributo: Seguridad / Confidencialidad

Campo SEI/Bass	Valor
Fuente del estímulo	Actor externo (ataque man-in-the-middle en red pública)
Estímulo	Intento de interceptar datos financieros del cliente durante el flujo de consentimiento y consulta Open Finance
Ambiente	Sistema operando normalmente, usuario en flujo activo de cotización de seguro de vida hipotecario
Artefacto	Adaptador Open Finance + capa de transporte + servicio de tokenización de PII
Respuesta	El sistema transmite todos los datos financieros cifrados con TLS 1.3 y tokeniza los campos PII antes de cualquier transmisión; solo los servicios autorizados con clave KMS pueden operar sobre los datos originales
Medida de respuesta	100% de las transmisiones Open Finance usan TLS 1.3; 100% de los campos PII son tokenizados antes de salir del servicio de perfilamiento; cero datos personales en texto plano en tránsito

Escenario narrativo:

Como actor externo (atacante) que intenta un ataque man-in-the-middle en una red pública, cuando intenta interceptar los datos financieros del cliente durante el flujo de consentimiento y consulta Open Finance, dado que el ambiente es un sistema operando normalmente con el usuario en un flujo activo de cotización de seguro de vida hipotecario, quiero que el sistema transmita todos los datos financieros cifrados con TLS 1.3 y tokenice los campos PII antes de cualquier transmisión, esto debe suceder garantizando que el 100% de las transmisiones Open Finance usen TLS 1.3, el 100% de los campos PII estén tokenizados antes de salir del servicio de perfilamiento, y cero datos personales en texto plano en tránsito.

Historia de usuario en Jira:

HU-ARQ-03 — Tokenización de PII y Trazabilidad — <https://proyectointegradorgrupo2.atlassian.net/browse/SOL-23>

Dependencias:

- HU-ARQ-04 (SOL-22) — Gestión de consentimientos: sin consentimiento activo no se intercambian datos Open Finance
- FE-05 (SOL-10) — Autenticación biométrica y KYC: los datos del usuario se usan en el intercambio Open Finance

- HU-ARQ-08 (SOL-37) — Kafka: los eventos con PII deben estar cifrados end-to-end en los topics
- FE-01 (SOL-3) — Cotización: es el journey principal que activa el intercambio de datos Open Finance

ASR-4.2 — Seguridad — Integridad de pólizas emitidas en almacenamiento

Atributo: Seguridad / Integridad

Campo SEI/Bass	Valor
Fuente del estímulo	Actor interno con credenciales comprometidas
Estímulo	Intento de modificar el contenido de una póliza emitida y firmada digitalmente, almacenada en S3
Ambiente	Sistema en producción, póliza ya emitida con firma electrónica válida del cliente
Artefacto	Servicio de Emisión & Pólizas + Amazon S3 + AWS KMS + registro de auditoría
Respuesta	El sistema verifica la firma digital de la póliza en cada lectura; solo las modificaciones realizadas por el servicio de Emisión con token de autorización válido quedan registradas; cualquier intento de alteración directa sobre S3 es detectado y auditado
Medida de respuesta	100% de las modificaciones a pólizas emitidas son realizadas por servicios autorizados con token válido; 100% de los intentos de alteración no autorizada son detectados y registrados en el log de auditoría en < 1 segundo

Escenario narrativo:

Como actor interno con credenciales comprometidas, cuando se intenta modificar el contenido de una póliza emitida y firmada digitalmente almacenada en S3, dado que el ambiente es un sistema en producción con la póliza ya emitida y con firma electrónica válida del cliente, quiero que el sistema verifique la firma digital de la póliza en cada lectura y que cualquier intento de alteración directa sobre S3 sea detectado y auditado, esto debe suceder garantizando que el 100% de las modificaciones a pólizas emitidas provengan de servicios autorizados con token válido, y que el 100% de los intentos de alteración no autorizada sean detectados y registrados en el log de auditoría en menos de 1 segundo.

Historia de usuario en Jira:

HU-ARQ-11 — Integridad y Auditoría de Pólizas Emitidas —
<https://proyectointegradorgroupo2.atlassian.net/browse/SOL-45>

Dependencias:

- FE-02 (SOL-4) — Suscripción y emisión: la póliza emitida es el artefacto protegido por este ASR
- HU-ARQ-03 (SOL-23) — Tokenización PII: la firma digital de la póliza también protege campos PII embebidos
- ASR-4.1 (SOL-23) — confidencialidad: la integridad y la confidencialidad son complementarias en el mismo documento
- HU-ARQ-04 (SOL-22) — Consentimientos: el log de auditoría de consentimientos usa la misma infraestructura S3 Object Lock

Sección 2: Catálogo de Historias de Usuario

Listado completo de las 20 historias del backlog con enlace directo a Jira. Para ver criterios de aceptación, descripción completa y ASR asociado, consultar directamente la historia en el tablero.

ID	Nombre de la historia	Enlace en Jira
SOL-3	Cotización de seguros en tiempo real	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-3
SOL-4	Suscripción y emisión de póliza	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-4
SOL-5	Administración de pólizas	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-5
SOL-6	Gestión y seguimiento de siniestros	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-6
SOL-10	Autenticación biométrica y KYC/AML	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-10
SOL-11	Billetera de pólizas móvil (offline)	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-11
SOL-13	Notificaciones y asistencia geolocalizada	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-13
SOL-12	Reporte de siniestros con evidencia multimedia	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-12
SOL-31	Portal de socios distribuidores (API B2B)	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-31
SOL-21	Parametrización Multitenant / Multirregión	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-21
SOL-20	Abstracciones de Pasarelas de Pago (Adapter)	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-20
SOL-23	Tokenización de PII y Trazabilidad	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-23
SOL-22	Gestión de Consentimientos Open Finance	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-22
SOL-26	Persistencia PostgreSQL + Redis	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-26
SOL-29	Tolerancia a Fallos y Failover (Circuit Breaker)	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-29
SOL-28	Optimización de Latencia Motor de Scoring	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-28
SOL-37	Event Bus Kafka y Degradación Elegante	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-37
SOL-43	Autoescalado Horizontal EKS (HPA)	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-43

ID	Nombre de la historia	Enlace en Jira
SOL-44	Offline Sync BFF Móvil	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-44
SOL-45	Integridad y Auditoría de Pólizas Emitidas	https://proyectointegradorgrupo2.atlassian.net/browse/SOL-45

Tablero Jira del proyecto: <https://proyectointegradorgrupo2.atlassian.net>